<Bolsa Futuro Digital>

<aula 25>

Turma de FrontEnd - IF UFRGS

JSON

JSON significa **JavaScript Object Notation** — uma forma de escrever dados que tanto humanos quanto computadores conseguem ler facilmente.

É o formato mais usado para **enviar** e **receber dados** entre sistemas — por exemplo, entre o navegador e um servidor. Parece muito com um objeto JavaScript, mas tem **regras mais rígidas**.

```
// Objeto JavaScript (o que você já conhece)
let usuario = {
  nome: 'Ana',
  idade: 20,
  ativo: true
}

// O mesmo dado em JSON (é uma string!)
let useuarioJson = {
    "nome":"Ana",
    "idade":20,
    "ativo":true
}

// JSON é sempre texto —
// chaves e strings usam aspas duplas.
```

JSON

JSON aceita apenas 6 tipos de dados:

- **string** – texto em aspas duplas
- **number** – inteiro ou decimal
- **boolean** – true ou false
- **null** – valor vazio
- **array** – lista de valores
- **object** – conjunto chave/valor

Não existem em JSON: undefined, Date, funções ou Symbol.

```
let user = {
  "nome": "Lucas",              // string
  "idade": 21,                  // number
  "aprovado": true,             // boolean
  "nota": null,                 // null
  "cursos": [                   // array
    "HTML",
    "CSS",
    "JavaScript"
  ],
  "endereco": {                 // object
    "cidade": "Porto Alegre",
    "uf": "RS"
  }
}
```

JSON

JSON é mais rígido que JavaScript. Regras principais:

- Chaves e strings **sempre** entre **aspas duplas**
- Sem vírgula no **último item**
- Sem comentários
- Sem **undefined**, funções ou Symbol como valor

Um JSON inválido causa um **erro** ao ser lido pelo JavaScript.

```
// JSON inválido
let user1 = {
  nome: 'Ana',          // chave sem aspas duplas
  "cidade": 'SP',       // aspas simples na string
  "ativo": undefined,   // undefined não existe
  "cursos": [],         // vírgula sobrando aqui
}

// JSON válido
let user2 = {
  "nome": "Ana",
  "cidade": "SP",
  "ativo": true,
  "cursos": []
}
```

JSON

O JavaScript tem dois métodos nativos:

JSON.stringify()
Transforma um objeto JS em uma string JSON. Usado para enviar dados.

JSON.parse()
Transforma uma string JSON de volta em objeto JS. Usado para receber dados.

```
let produto = {
  nome: 'Teclado',
  preco: 350,
  emEstoque: true
}

// Objeto → String JSON
let texto = JSON.stringify(produto)

// String JSON → Objeto
let obj = JSON.parse(texto)
```

API

API significa **Application Programming Interface** — uma interface que permite dois sistemas se **comunicarem**.

É como um **garçom**: você (navegador) faz um pedido, o garçom (API) leva ao cozinheiro (servidor), e traz a resposta de volta.

Na web, APIs trocam dados em formato **JSON** pelo protocolo **HTTP**.



Backend



API



Frontend

Tipos de API

APIs Públicas

Disponíveis para qualquer pessoa, sem autenticação ou com chave gratuita de acesso.

```
// Exemplos reais:
ViaCEP      → endereço por CEP
OpenWeather → clima e temperatura
IBGE        → dados do Brasil
PokeAPI     → dados de pokémon
GitHub      → repositórios e
              usuários
```

APIs Privadas

Criadas por empresas para uso interno ou parceiros. Exigem autenticação para acessar.

```
// Exemplos reais:
API do banco → saldo e transações
API do app   → perfil do usuário
API do ERP   → estoque e pedidos
```

Comunicação com API

Toda comunicação segue o modelo **Requisição → Resposta**:

1. O navegador envia uma **requisição HTTP** para uma **URL**
2. A URL identifica o **recurso** desejado
3. O servidor processa e devolve uma **resposta**

A resposta traz um **status** code:
200 = OK | 404 = não encontrado | 500 = erro no servidor



Verbos HTTP

Cada requisição HTTP usa um verbo que indica a intenção da operação sobre um recurso:

Verbo	O que faz	Exemplo de uso
GET	Buscar / ler dados	Listar usuários, buscar um produto pelo ID
POST	Criar um novo dado	Cadastrar um usuário, criar um pedido
PUT	Substituir um dado inteiro	Atualizar todos os campos de um perfil
PATCH	Atualizar parcialmente	Alterar só o e-mail de um usuário
DELETE	Apagar um dado	Remover um comentário ou post

Rest - Conceitos

REST é um conjunto de **convenções** para organizar APIs de forma consistente e previsível.

A ideia central: cada **URL representa um recurso**, e o **verbo HTTP** determina o que fazer com ele.

Uma API que segue essas regras é chamada de **RESTful**.

```
// Recurso: /usuarios

GET    /usuarios        → lista todos os usuários
GET    /usuarios/42     → busca o usuário 42
POST   /usuarios        → cria um novo usuário
PUT    /usuarios/42     → substitui o usuário 42
PATCH  /usuarios/42     → atualiza parte do 42
DELETE /usuarios/42     → apaga o usuário 42

// Recurso aninhado: pedidos de um usuário

GET    /usuarios/42/pedidos    → pedidos do 42
GET    /usuarios/42/pedidos/7  → pedido 7 do 42
```

Promise

Uma **Promise** representa uma operação que **ainda não terminou**, mas vai terminar no futuro.

Ela pode estar em três estados:

- **pending** — ainda aguardando
- **fulfilled** — concluída com sucesso
- **rejected** — concluída com erro

É o mecanismo que o JavaScript usa para lidar com operações **assíncronas** — como buscar dados de uma API.

```
// Analogia: delivery de comida
// "Vou te entregar em 30 min" → pending
// Pedido chegou ✓             → fulfilled
// Pedido cancelado ✗          → rejected

// fetch() retorna uma Promise:
let promessa = fetch('https://viacep.com.br/ws/
01001000/json')

console.log(promessa)
// Promise { <pending> }
// (o resultado ainda não chegou!)

// Para usar o resultado, precisamos
// de .then() ou async/await.
```

Promise

Criamos uma Promise com **new Promise()**. Ela recebe uma função com dois parâmetros:

- **resolve** – chame quando der certo
- **reject** – chame quando der errado

Na prática você raramente cria Promises na mão – o fetch já faz isso. Mas entender o mecanismo ajuda a compreender como tudo funciona.

```
function esperar(segundos) {
  return new Promise(function(resolve, reject) {

    if (segundos < 0) {
      reject('Número de segundos inválido')
      return
    }

    setTimeout(function() {
      resolve('Pronto! Esperei ' + segundos +
      's')
    }, segundos * 1000)

  })
}

esperar(2)  // retorna uma Promise
```

Métodos then() e catch()

Para usar o resultado de uma Promise, encadeamos métodos:

.then() – executado quando a Promise é **resolvida** (sucesso). Pode ser encadeado várias vezes.

.catch() – executado quando a Promise é **rejeitada** (erro). Captura erros de todos os .then() anteriores.

```
esperar(2)
  .then(function(mensagem) {
    console.log(mensagem) // 'Pronto! Esperei 2s'
    return 'próximo passo'
  })
  .then(function(valor) {
    console.log(valor)    // 'próximo passo'
  })
  .catch(function(erro) {
    console.log('Deu erro:', erro)
  })

// fetch com then/catch:
fetch('https://viacep.com.br/ws/01001000/json')
  .then(function(res)   { return res.json() })
  .then(function(dados) { console.log(dados.logradouro) })
  .catch(function(erro)  { console.log('Erro:', erro) })
```

Promise.all() e Promise.race()

```
Promise.all()
/*
Espera todas as Promises
terminarem. Retorna um array
com todos os resultados.

Se qualquer uma falhar,
o .all() falha também.
*/
let p1 = fetch('/api/usuario')
let p2 = fetch('/api/pedidos')

Promise.all([p1, p2])
  .then(function(resultados) {
    // resultados[0] → usuário
    // resultados[1] → pedidos
  })
  .catch(function(erro) {
    console.log(erro)
  })
```

```
Promise.race()
/*
Retorna o resultado da
primeira Promise que
terminar (sucesso ou erro).

Útil para criar timeout:
*/
function timeout(ms) {
  return new Promise(function(_, reject) {
    setTimeout(function() {
      reject('tempo esgotado')
    }, ms)
  })
}

Promise.race([
  fetch('/api/dados'),
  timeout(5000)
])
  .then(function(res) { /* ... */ })
  .catch(function(e) { console.log(e) })
```

Async/Await

async/await é uma forma mais **legível** de trabalhar com Promises — o código parece sequencial, mas continua sendo assíncrono.

async — declara que a função vai trabalhar com Promises. Ela passa a **retornar uma Promise** automaticamente.

await — pausa a execução daquela função até a Promise resolver, sem travar o navegador.

```
// Com .then() — encadeamento
fetch('/api/usuario')
  .then(function(res)   { return res.json() })
  .then(function(dados) { console.log(dados.nome) })
  .catch(function(erro)  { console.log(erro) })

// Com async/await — mais legível!
async function buscarUsuario() {
  let res   = await fetch('/api/usuario')
  let dados = await res.json()
  console.log(dados.nome)
}

buscarUsuario()
// Mesmo resultado — código mais claro!
```

Async/Await

Regras importantes:

- **await** só pode ser usado dentro de uma função **async**
- Uma função **async** sempre retorna uma **Promise**
- Se a Promise for rejeitada, o await **lança um erro**
- Sem **try/catch**, esse erro propaga e pode travar a aplicação

Sempre use **try/catch** dentro de funções async!

```
// await fora de async — SyntaxError!
let res = await fetch('/api')

// Correto: dentro de função async
async function buscar() {
  let res = await fetch('/api')  // ✓
  return res.json()
}

// Resultado de função async é Promise:
let resultado = buscar()
console.log(resultado)  // Promise { <pending> }

// Para obter o valor, use await:
async function principal() {
  let dados = await buscar()
  console.log(dados)
}
```

Fetch com Async/Await e try/cath

```
async function buscarEndereco(cep) {
  try {

    // 1. Faz a requisição para a API
    let resposta = await fetch('https://viacep.com.br/ws/' + cep + '/json')

    // 2. Verifica se o servidor retornou OK (status 200-299)
    if (!resposta.ok) {
      throw new Error('CEP não encontrado. Status: ' + resposta.status)
    }

    // 3. Converte a resposta de JSON string para objeto JavaScript
    let dados = await resposta.json()

    // 4. Usa os dados
    console.log(dados.logradouro + ', ' + dados.localidade + ' - ' + dados.uf)

  } catch (erro) {
    // Captura qualquer falha: sem internet, CEP inválido, servidor fora do ar...
    console.log('Erro ao buscar CEP:', erro.message)
  }
}

buscarEndereco('01310100')
// Avenida Paulista, São Paulo - SP
```

Exercícios

Exercício 1: Crie um objeto com os dados de um produto (nome, preco, categoria). Use JSON.stringify para transformá-lo em texto e JSON.parse para transformar o texto de volta em objeto. Exiba ambos no console e compare os tipos com typeof.

Exercício 2: Use fetch com .then() e .catch() para buscar o endereço do seu CEP na API ViaCEP (viacep.com.br/ws/'seu_cep'/json). Exiba no console o logradouro e o bairro.

Exercício 3: Reescreva o exercício 2 usando async/await com try/catch, dentro de uma função chamada buscarCep(). Se o servidor retornar erro (resposta.ok === false), exiba uma mensagem amigável.

</aula 25>